Name: J.Chandra hasini
Roll_no: 2520030086
Sub: OSSP practical 7 & 8

7. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Program:

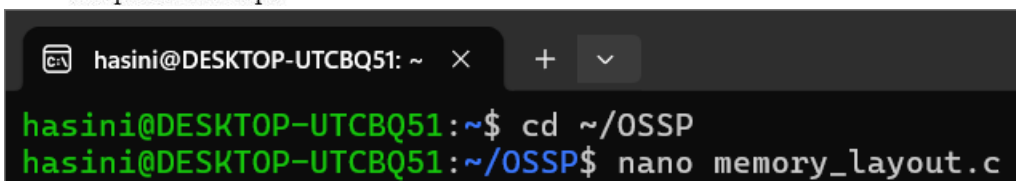
```
#include <stdio.h>
#include <stdlib.h>
int global=100;
static int s=50;

int main()
{
   int stack=10;
   int *heap=(int*)malloc(sizeof(int));

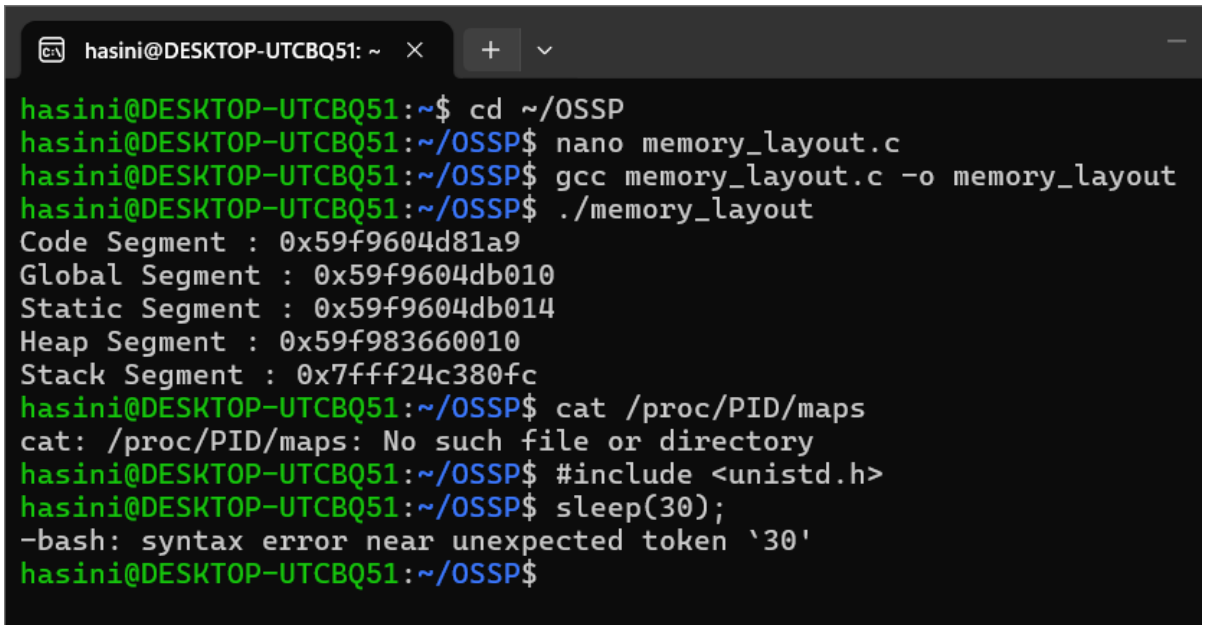
   printf("Code Segment _: %p\n",main);
   printf("Global Segment : %p\n",&global);
   printf("Static Segment : %p\n",&s);
   printf("Heap Segment _: %p\n",heap);
   printf("Stack Segment _: %p\n",&stack);
   free(heap);
   return 0;
}
```

Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

ps
cat /proc/PID/maps




hasini@DESKTOP-UTCBQ51: ~
GNU nano 8.7.1 memory_layout.c *
#include <stdio.h>
#include <stdlib.h>
int global = 100;
static int s = 50;
int main()
{
int stack = 10;
int *heap = (int *)malloc(sizeof(int));
printf("Code Segment : %p\n", (void *)main);
printf("Global Segment : %p\n", (void *)&global);
printf("Static Segment : %p\n", (void *)&s);
printf("Heap Segment : %p\n", (void *)heap);
printf("Stack Segment : %p\n", (void *)&stack);
free(heap);
return 0;
}
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify
hasini@DESKTOP-UTCBQ51:~/OSSP$ gcc memory_layout.c -o memory_layout
hasini@DESKTOP-UTCBQ51:~/OSSP$ ./memory_layout
Code Segment : 0x644882b851a9
Global Segment : 0x644882b88010
Static Segment : 0x644882b88014
Heap Segment : 0x6448bd515010
Stack Segment : 0x7ffdd0d1917c
hasini@DESKTOP-UTCBQ51:~/OSSP$


hasini@DESKTOP-UTCBQ51: ~
hasini@DESKTOP-UTCBQ51:~$ cd ~/OSSP
hasini@DESKTOP-UTCBQ51:~/OSSP$ nano memory_layout.c
hasini@DESKTOP-UTCBQ51:~/OSSP$ gcc memory_layout.c -o memory_layout
hasini@DESKTOP-UTCBQ51:~/OSSP$ ./memory_layout
Code Segment : 0x59f9604d81a9
Global Segment : 0x59f9604db010
Static Segment : 0x59f9604db014
Heap Segment : 0x59f983660010
Stack Segment : 0x7fff24c380fc
hasini@DESKTOP-UTCBQ51:~/OSSP$ cat /proc/PID/maps
cat: /proc/PID/maps: No such file or directory
hasini@DESKTOP-UTCBQ51:~/OSSP$ #include <unistd.h>
hasini@DESKTOP-UTCBQ51:~/OSSP$ sleep(30);
-bash: syntax error near unexpected token `30'
hasini@DESKTOP-UTCBQ51:~/OSSP$

8A

8. Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Program:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int *a,*b;

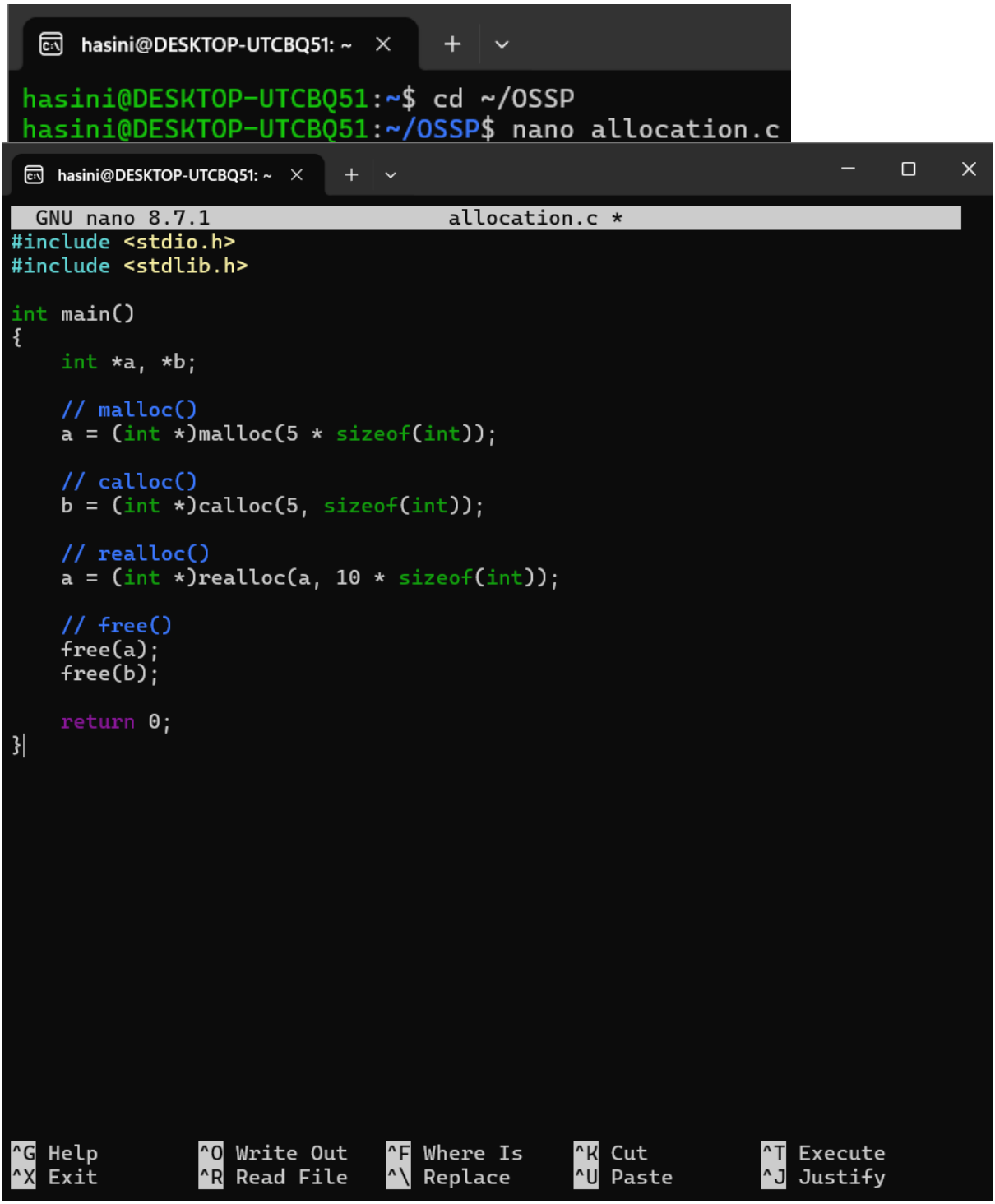
   a=(int*)malloc(5*sizeof(int));

   b=(int*)calloc(5,sizeof(int));

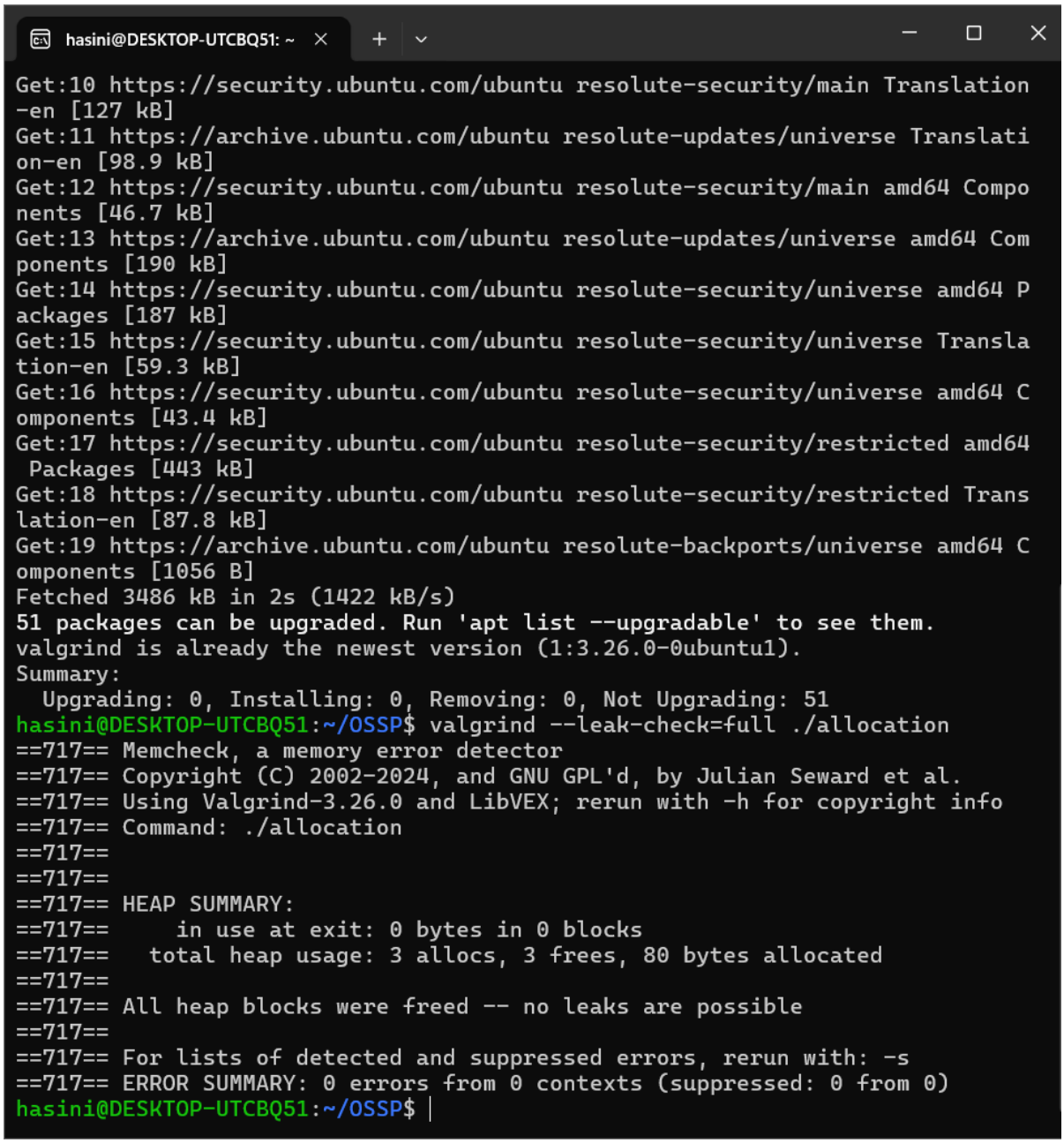
   a=(int*)realloc(a,10*sizeof(int));

   free(a);
   free(b);

   return 0;
}
```


hasini@DESKTOP-UTCBQ51: ~
hasini@DESKTOP-UTCBQ51:~$ cd ~/OSSP
hasini@DESKTOP-UTCBQ51:~/OSSP$ nano allocation.c
hasini@DESKTOP-UTCBQ51: ~
GNU nano 8.7.1 allocation.c *
#include <stdio.h>
#include <stdlib.h>
int main()
{
int *a, *b;
// malloc()
a = (int *)malloc(5 * sizeof(int));
// calloc()
b = (int *)calloc(5, sizeof(int));
// realloc()
a = (int *)realloc(a, 10 * sizeof(int));
// free()
free(a);
free(b);
return 0;
}
^G Help
^O Write Out
^F Where Is
^K Cut
^T Execute
^X Exit
^R Read File
^\ Replace
^U Paste
^J Justify

```
hasini@DESKTOP-UTCBQ51:~/OSSP$ gcc allocation.c -o allocation
hasini@DESKTOP-UTCBQ51:~/OSSP$ ./allocation
hasini@DESKTOP-UTCBQ51:~/OSSP$ valgrind --leak-check=full ./allocation
==547== Memcheck, a memory error detector
==547== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==547== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==547== Command: ./allocation
==547==
==547==
==547== HEAP SUMMARY:
==547==     in use at exit: 0 bytes in 0 blocks
==547==   total heap usage: 3 allocs, 3 frees, 80 bytes allocated
==547==
==547== All heap blocks were freed -- no leaks are possible
==547==
==547== For lists of detected and suppressed errors, rerun with: -s
==547== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
hasini@DESKTOP-UTCBQ51:~/OSSP$ sudo apt update
sudo apt install valgrind
[sudo: authenticate] Password:
Hit:1 https://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 https://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB
]
Get:3 https://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 https://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB
]
Get:5 https://security.ubuntu.com/ubuntu resolute-security/main amd64 Packag
es [543 kB]
Get:6 https://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages
 [679 kB]
Get:7 https://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en
 [162 kB]
Get:8 https://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Componen
ts [98.0 kB]
Get:9 https://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Pack
ages [308 kB]
Get:10 https://security.ubuntu.com/ubuntu resolute-security/main Translation
-en [127 kB]
Get:11 https://archive.ubuntu.com/ubuntu resolute-updates/universe Translati
```


hasini@DESKTOP-UTCBQ51: ~
Get:10 https://security.ubuntu.com/ubuntu resolute-security/main Translation
-en [127 kB]
Get:11 https://archive.ubuntu.com/ubuntu resolute-updates/universe Translati
on-en [98.9 kB]
Get:12 https://security.ubuntu.com/ubuntu resolute-security/main amd64 Compo
nents [46.7 kB]
Get:13 https://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Com
ponents [190 kB]
Get:14 https://security.ubuntu.com/ubuntu resolute-security/universe amd64 P
ackages [187 kB]
Get:15 https://security.ubuntu.com/ubuntu resolute-security/universe Transla
tion-en [59.3 kB]
Get:16 https://security.ubuntu.com/ubuntu resolute-security/universe amd64 C
omponents [43.4 kB]
Get:17 https://security.ubuntu.com/ubuntu resolute-security/restricted amd64
Packages [443 kB]
Get:18 https://security.ubuntu.com/ubuntu resolute-security/restricted Trans
lation-en [87.8 kB]
Get:19 https://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 C
omponents [1056 B]
Fetched 3486 kB in 2s (1422 kB/s)
51 packages can be upgraded. Run 'apt list --upgradable' to see them.
valgrind is already the newest version (1:3.26.0-0ubuntu1).
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 51
hasini@DESKTOP-UTCBQ51:~/OSSP$ valgrind --leak-check=full ./allocation
==717== Memcheck, a memory error detector
==717== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==717== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==717== Command: ./allocation
==717==
==717==
==717== HEAP SUMMARY:
==717== in use at exit: 0 bytes in 0 blocks
==717== total heap usage: 3 allocs, 3 frees, 80 bytes allocated
==717==
==717== All heap blocks were freed -- no leaks are possible
==717==
==717== For lists of detected and suppressed errors, rerun with: -s
==717== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
hasini@DESKTOP-UTCBQ51:~/OSSP$

8B

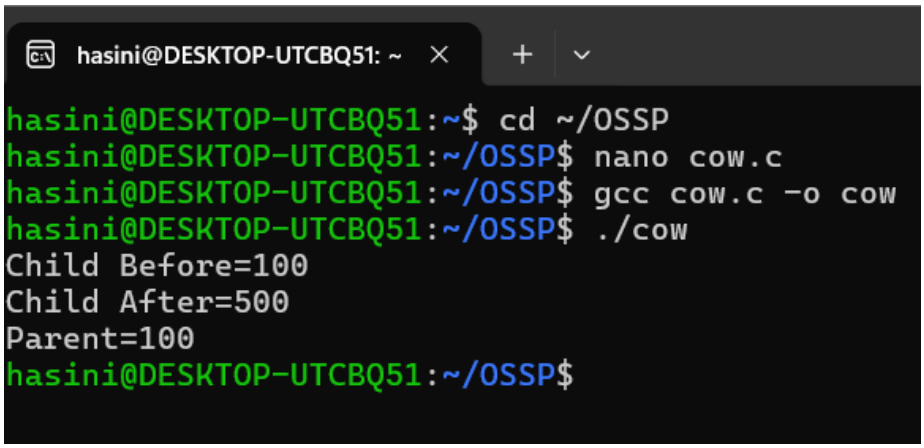
Run valgrind and check the result as: All heap blocks were freed -- no leaks are possible.

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
#include <stdio.h>
#include <unistd.h>
int main()
{
   int x=100;
   if(fork()==0)
   {
      printf("Child Before=%d\n",x);
      x=500;
      printf("Child After=%d\n",x);
   }
   else
   {
      sleep(2);
      printf("Parent=%d\n",x);
   }

   return 0;
}
```


hasini@DESKTOP-UTCBQ51: ~
GNU nano 8.7.1 cow.c *
#include <stdio.h>
#include <unistd.h>
int main()
{
int x = 100;
if(fork() == 0)
{
printf("Child Before=%d\n", x);
x = 500;
printf("Child After=%d\n", x);
}
else
{
sleep(2);
printf("Parent=%d\n", x);
}
return 0;
}
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify


hasini@DESKTOP-UTCBQ51: ~
hasini@DESKTOP-UTCBQ51:~$ cd ~/OSSP
hasini@DESKTOP-UTCBQ51:~/OSSP$ nano cow.c
hasini@DESKTOP-UTCBQ51:~/OSSP$ gcc cow.c -o cow
hasini@DESKTOP-UTCBQ51:~/OSSP$ ./cow
Child Before=100
Child After=500
Parent=100
hasini@DESKTOP-UTCBQ51:~/OSSP$